Preference-Aware Event Seating Optimization Using Constraint Satisfaction and Backtracking Algorithms

Gudipudi Aneesha, Scholar, M.Tech
Department of Computer Science and Engineering, Koneru Lakshmaiah Education Foundation
2601050114@kluniversity.in

Raju Anitha, Professor
Department of Computer Science and Engineering, Koneru Lakshmaiah Education Foundation
anitharaju@kluniversity.in

***Abstract*—**Event seating is a combinatorial assignment problem in which organizers must allocate guests to seats while satisfying capacity, accessibility, group, adjacency, and separation requirements. Manual planning becomes difficult as the guest list and the number of preferences increase. This paper presents a preference-aware event seating optimization framework that models seating as a Constraint Satisfaction Problem (CSP) and uses backtracking to construct feasible assignments. Guests are represented as variables, candidate seats or tables form their domains, and mandatory requirements are represented as hard constraints. Optional preferences are represented as weighted soft constraints and evaluated using a preference score. The design includes constraint checking, recursive search, and optional heuristics such as minimum remaining values, degree ordering, least-constraining value, and forward checking. A Flask-based web application is proposed to collect event details and display seating assignments. The paper defines an evaluation protocol based on feasibility, hard-constraint violations, weighted preference satisfaction, runtime, and search nodes. Since measured implementation results are not available in the supplied project material, this paper does not claim empirical performance; it provides a reproducible design and evaluation plan for implementation and validation.

***Index Terms*—**Constraint satisfaction problem, backtracking, event seating, soft constraints, preference optimization, heuristic search, Flask.

I. INTRODUCTION

Seating assignment is an important operational task in weddings, conferences, academic functions, corporate events, and social gatherings. An organizer may need to seat family members together, keep specified guests apart, reserve accessible seats, respect table capacities, and place selected guests near a host. These requirements interact: satisfying one preference may restrict the choices available for several other guests. The task is therefore naturally represented as a constrained combinatorial problem rather than a simple list of independent assignments.

Constraint satisfaction provides a systematic representation of variables, domains, and restrictions [1]–[4], [7]. Backtracking builds a candidate solution incrementally and reverses a choice when it prevents completion. Improvements such as variable ordering, constraint propagation, and backjumping can reduce avoidable search [8], [9], [16], [22]. Preference modeling adds a second objective: among feasible arrangements, the system should favor those that satisfy more important optional requests [5].

Recent work on social classroom seating formulates seating as an optimization problem involving relationships, seat structure, and individual needs, and evaluates exact formulations and heuristics on realistic instances [1]. Motivated by this line of research, this paper describes a practical, web-oriented framework for event seating. The intended contribution is the integration of hard-constraint validation, weighted soft-preference scoring, recursive search, and a user-facing interface. It is a design and implementation paper; claims about speed or superiority require experiments on a completed implementation.

A. Problem Statement

Given a set of guests, a set of tables and their capacities, and a set of hard constraints and soft preferences, generate a seating assignment that satisfies every hard constraint and maximizes the weighted satisfaction of soft preferences. If no assignment satisfies all hard constraints, the system should report infeasibility rather than return an invalid plan.

B. Objectives and Contributions

The objectives are to: (1) define a CSP representation for guest seating; (2) implement backtracking with immediate hard-constraint checks; (3) distinguish mandatory requirements from preferences; (4) define a transparent preference score; (5) outline a Flask-based workflow; and (6) establish tests and metrics for reproducible evaluation. The paper contributes a clear modeling and evaluation specification suitable for a prototype, not a claim that the proposed solver is novel or experimentally optimal.

II. RELATED WORK

CSP research has developed the central concepts used by the proposed model. Montanari studied networks of constraints [6], while Mackworth formalized consistency in relational networks [7]. Kumar and Nadel surveyed CSP algorithms and their design considerations [2], [3]. Brailsford et al. reviewed CSP algorithms and applications, emphasizing their practical reach across scheduling and allocation tasks [4]. Meseguer provided an overview of CSP concepts [5]. These works motivate representing a seating instance through explicit variables, domains, and constraints.

Search efficiency is a major theme in CSP research. Dechter and Pearl studied network-based heuristics [8]. Dechter examined backjumping, learning, and cutset decomposition as enhancement schemes [22], and Dechter and Frost analyzed backjump-based backtracking and combinations of look-back and look-ahead techniques [9]. Later surveys describe developments in CSP, planning, and scheduling [16], while constraint-logic-programming and constraint-based scheduling surveys demonstrate the use of constraints in operational systems [20], [21].

Preference handling is distinct from feasibility. Rossi, Venable, and Walsh explain how soft constraints can represent quantitative and qualitative preferences [5]. For seating, a preference such as “sit near a colleague” should contribute to solution quality but must not override mandatory accessibility or separation rules. The distinction between hard and soft constraints is central to the proposed framework.

Seating also relates to assignment, placement, graph coloring, and scheduling. Surveys of quadratic assignment problems discuss objective functions that depend on relationships between assigned entities and locations [10]–[12]. Taillard and subsequent work on large-neighborhood search illustrate the need for stronger search strategies in large assignment problems [13]–[15]. Graph-coloring approaches have been applied to workforce and timetable assignment [18], [19], [23], [24]. Resource-constrained scheduling surveys provide additional context for allocation under limited capacity [17]. Hill et al. directly study social classroom seating

assignment and account for social relations, physical seating structure, and special needs [1].

A. Research Gap

The cited literature offers established models and algorithms for constraints, preferences, assignment, and scheduling. The project report identifies a practical integration need: a lightweight interface that accepts guest and table information, validates rules, invokes a solver, and explains the resulting plan. The proposed system addresses this engineering need. A formal novelty claim would require a more exhaustive comparison with existing event-seating applications and algorithms.

III. PROBLEM FORMULATION

A. Sets and Variables

Let $G = \{g_1, \ldots, g_n\}$ be the set of guests and $T = \{t_1, \ldots, t_k\}$ be the set of tables. Each table t_j has capacity c_j. Define $x_{ij} = 1$ if guest g_i is assigned to table t_j, and 0 otherwise. A seat-level model can be used when individual physical seats and adjacency are important; a table-level model is sufficient for table grouping and separation constraints.

B. Hard Constraints

Every guest must be assigned exactly once: $\Sigma_j x_{ij} = 1$ for each guest i. Table capacity must be respected: $\Sigma_i x_{ij} \leq c_j$ for each table j. For a pair of guests (i, l) that must not share a table, $x_{ij} + x_{lj} \leq 1$ for every table j. Accessibility requirements restrict the domain of a guest to compatible seats or tables. Duplicate identifiers and invalid capacities are rejected during input validation. These rules define feasibility and are never traded against preference scores.

C. Soft Preferences

Soft preferences may express that two guests prefer the same table, a guest prefers proximity to the host, or members of a group prefer to be near one another. Let $P = \{p_1, \ldots, p_m\}$ be the set of preferences, $w_r \geq 0$ the weight of preference r, and $s_r(X) \in \{0,1\}$ indicate whether arrangement X satisfies it. The weighted score is:

$$S(X) = \Sigma_{r=1}^{m} w_r s_r(X) \qquad (1)$$

The normalized preference score is $100 \times S(X)/\Sigma_r w_r$ when the maximum possible weight is positive. If all weights are zero, the score should be reported as not applicable rather than dividing by zero. The optimization objective is to maximize S(X) subject to all hard constraints. A first-feasible backtracking solver may return a feasible solution without proving that its preference score is optimal; preference maximization requires continued search, branch-and-bound, or another optimization strategy.

D. Assumptions

The initial prototype assumes a single event, finite guest and table lists, known capacities, and preferences supplied by the organizer. It does not infer sensitive relationships or automatically decide which preferences are more important. The organizer is responsible for confirming constraints and weights. Physical adjacency is only modeled if the venue layout is represented explicitly.

IV. PROPOSED SYSTEM ARCHITECTURE

The architecture consists of five components: (1) a web interface for event, guest, table, and preference entry; (2) a Flask controller that validates requests; (3) a CSP model that stores variables, domains, and constraints; (4) a backtracking solver; and (5) a results view showing assignments, capacity usage, preference score, and warnings. HTML and CSS provide the layout, while JavaScript may support dynamic guest entry and client-side validation. Solver validation remains authoritative on the server.

A. Workflow

The organizer enters guests and tables, then specifies mandatory rules and optional preferences. The application checks input integrity and verifies that total capacity is at least the number of guests. The solver initializes candidate domains, selects an unassigned guest, tries a candidate table or seat, and checks all affected hard constraints. If the partial assignment is valid, search continues recursively. If no candidate works, the last assignment is removed and the next candidate is tried. Once a complete feasible assignment is found, the application calculates the preference score and renders the result. If all branches fail, it returns an explicit infeasibility message.

B. Software Design

A modular implementation can separate app.py (Flask routes), models.py (guest, table, and preference structures), constraints.py (hard-constraint validation), optimizer.py (search and scoring), and templates for input and results. A separate test module should exercise the solver without requiring the web interface. The development stack specified in the project report is Python 3.10 or later, Flask, HTML5, CSS3, JavaScript, and a modern browser.

C. Privacy and Responsible Use

Guest lists may contain personal information. The prototype should collect only data needed for seating, avoid exposing guest details in public URLs or logs, and restrict access if deployed for real events. Synthetic guest records should be used for development tests. Accessibility requirements should be treated as functional constraints, and organizers should review the final seating chart before use.

V. BACKTRACKING SOLVER

A. Basic Algorithm

Backtracking explores a search tree of partial assignments. At each level it chooses an unassigned guest and tries each remaining candidate. A candidate is accepted only if it is consistent with the current partial assignment. Failure at a deeper level causes the algorithm to undo the most recent choice and continue with another candidate. The following pseudocode describes feasibility search.

```
BACKTRACK(A):
1. If every guest is assigned, return A.
2. Select an unassigned guest g.
3. For each candidate seat/table v in Domain(g):
   a. If A ∪ {g=v} satisfies hard constraints:
      i. Add g=v to A.
      ii. result ← BACKTRACK(A).
      iii. If result succeeds, return result.
      iv. Remove g=v from A.
4. Return FAILURE.
```

B. Search Heuristics

Minimum Remaining Values (MRV) chooses the unassigned guest with the fewest currently legal options. The degree heuristic breaks ties by selecting a guest constrained by the largest number of other unassigned guests. Least-Constraining Value (LCV) tries the candidate that removes the fewest options from neighboring guests. Forward checking updates remaining domains after each assignment and detects some dead ends early. These methods are established CSP strategies [2], [3], [8], [9], [16]. Their impact must be measured for the seating model rather than assumed.

C. Preference Optimization

For small instances, the solver can explore all feasible assignments and retain the assignment with the highest weighted score. For larger instances, branch-and-bound can maintain the best score found and an upper bound on the score achievable by a partial assignment. Search may stop at a time limit and return the best feasible arrangement found, but the interface must distinguish a

proven optimum from a best-known solution. Backjumping or constraint learning may be investigated if ordinary backtracking explores too many repeated dead ends [9], [22].

D. Complexity

With n guests and at most k candidate tables per guest, an unpruned search tree can have $O(k^n)$ leaves in a simplified model. This is a worst-case upper-bound illustration; capacities, domain reduction, and constraint propagation can substantially change actual search effort. Preference optimization may require examining multiple feasible assignments and can be more expensive than finding the first feasible arrangement. Runtime should therefore be measured together with search-node counts.

VI. IMPLEMENTATION AND EVALUATION PLAN

A. Input and Output

Inputs include event name, unique guest identifiers, table identifiers and capacities, together/separate preferences, accessibility requirements, and optional weights. Outputs include a table-wise seating chart, number of assigned guests, unused capacity, weighted preference score, solver status, runtime, and search nodes. A solver failure should explain that no feasible arrangement was found under the entered constraints; it should not silently discard mandatory rules.

B. Test Design

Unit tests should cover capacity checking, duplicate identifiers, together/separate constraints, accessible-seat compatibility, score calculation, and infeasible instances. Integration tests should confirm that Flask receives input, invokes the solver, and renders success or failure correctly. Boundary tests should include zero guests, exact capacity, insufficient capacity, and contradictory mandatory constraints. The project report proposes test cases for all of these conditions.

C. Evaluation Metrics

Feasibility rate is the percentage of instances for which a valid assignment is returned. Every accepted arrangement must have zero hard-constraint violations. Weighted preference satisfaction is the achieved score divided by maximum possible weight, multiplied by 100. Execution time measures end-to-end solver duration, and search nodes measure the amount of search performed. Evaluation should also record timeout or failure status so that unsuccessful runs are not omitted from comparisons.

D. Experimental Protocol

Create repeatable synthetic instances with 8, 16, 24, and 32 guests, sufficient total capacity, and controlled densities of together and separation constraints. Compare basic backtracking against heuristic-enhanced backtracking using the same instances and preference weights. Repeat runs where runtime variability is relevant and report mean and dispersion, alongside feasibility and score. Store the random seed and instance configuration for reproducibility. The supplied project material contains no measured results, so the result table must be populated only after the code is implemented and run.

E. Results Reporting Template

For each instance size, report the solver variant, feasibility status, runtime in milliseconds, nodes visited, hard-constraint violations, and preference score. A fair comparison must use the same datasets and stopping conditions. If a time limit is applied, report it and identify whether the returned solution is optimal, feasible, or best-known.

VII. DISCUSSION

The CSP representation makes seating rules explicit and independently testable. Separating hard constraints from soft preferences prevents a high preference score from masking a safety, accessibility, or capacity violation. Backtracking is a suitable baseline because it is understandable and supports correctness checks, while MRV, LCV, and forward checking offer plausible ways to reduce search. Nevertheless, the exponential worst-case search space limits scalability, and heuristic effectiveness depends on the instance structure.

The formulation is intentionally modular. A table-level assignment is easier to implement but cannot represent every adjacency requirement. If the event requires guests to sit in neighboring physical seats, the model must define individual seat variables and a venue adjacency graph. Likewise, maximizing a weighted score requires optimization beyond simply stopping at the first feasible assignment. These design decisions should be stated in the implementation and reflected in the evaluation.

The proposed system should be viewed as a decision-support tool rather than an unquestionable authority. Organizers should inspect the result, particularly when preferences conflict or requirements are incomplete. Future versions may compare backtracking with integer programming, local search, or large-neighborhood methods used in assignment research [10]–[15].

VIII. LIMITATIONS AND FUTURE WORK

The current design has several limitations. Backtracking can become expensive as guest count and domain size grow. Preference weights are subjective, and a single weighted sum may conceal trade-offs among different groups. The basic model assumes accurate input and may not capture venue geometry unless adjacency is explicitly represented. The web prototype also requires further work for authentication, persistent event storage, concurrent use, and secure handling of personal information.

Future work includes a floor-plan editor, Excel/CSV import and export, printable PDF seating charts, incremental re-optimization after guest changes, explanation of unsatisfied preferences, and evaluation on larger realistic datasets. Algorithmic extensions may include constraint learning, backjumping, branch-and-bound, integer programming, or neighborhood search. Any extension should be compared using reproducible instances and transparent metrics.

IX. CONCLUSION

This paper presents a preference-aware event seating framework based on CSP modeling and backtracking. Guests, candidate seats or tables, hard constraints, and weighted soft preferences are represented explicitly. The solver searches for feasible assignments and evaluates preference satisfaction without allowing soft preferences to override mandatory rules. A Flask-based architecture is outlined together with heuristics, complexity considerations, tests, and an evaluation protocol. The supplied project report does not include completed experimental measurements; accordingly, runtime and solution-quality claims remain to be validated. Implementation and controlled experiments are the necessary next steps before drawing conclusions about scalability or comparative performance.

REFERENCES

[1] A. Hill, D. Hom, S. Peuker, and I. Mourtos, “Social classroom seating assignment problems,” Networks, vol. 85, no. 2, pp. 166–188, 2025, doi: 10.1002/net.22252.

[2] V. Kumar, “Algorithms for constraint-satisfaction problems: A survey,” AI Magazine, vol. 13, no. 1, 1992.

[3] B. A. Nadel, “Constraint satisfaction algorithms: A survey,” Computational Intelligence, vol. 5, no. 3, pp. 188–224, 1989.

[4] S. C. Brailsford, C. N. Potts, and B. M. Smith, “Constraint satisfaction problems: Algorithms and applications,” European Journal of Operational Research, vol. 119, no. 3, pp. 557–581, 1999.

[5] F. Rossi, K. B. Venable, and T. Walsh, “Preferences in constraint satisfaction and optimization,” AI Magazine, vol. 29, no. 4, pp. 58–68, 2008, doi: 10.1609/aimag.v29i4.2202.

[6] U. Montanari, “Networks of constraints: Fundamental properties and applications to picture processing,” Information Sciences, vol. 7, no. 2, pp. 95–132, 1974.

[7] A. K. Mackworth, “Consistency in networks of relations,” Artificial Intelligence, vol. 8, no. 1, pp. 99–118, 1977, doi: 10.1016/0004-3702(77)90007-8.

[8] R. Dechter and J. Pearl, “Network-based heuristics for constraint-satisfaction problems,” Artificial Intelligence, vol. 34, no. 1, pp. 1–38, 1987.

[9] R. Dechter and D. Frost, “Backjump-based backtracking for constraint satisfaction problems,” Artificial Intelligence, vol. 136, no. 2, pp. 147–188, 2002, doi: 10.1016/S0004-3702(02)00120-0.

[10] E. M. Loiola, N. M. M. de Abreu, P. O. Boaventura-Netto, P. Hahn, and T. Querido, “A survey for the quadratic assignment problem,” European Journal of Operational Research, vol. 176, no. 2, pp. 657–690, 2007.

[11] M. Hanan and J. M. Kurtzberg, “A review of the placement and quadratic assignment problems,” SIAM Review, vol. 14, no. 2, pp. 324–342, 1972.

[12] R. E. Burkard, “Quadratic assignment problems,” European Journal of Operational Research, vol. 15, no. 3, pp. 283–289, 1984.

[13] É. D. Taillard, “Robust taboo search for the quadratic assignment problem,” Parallel Computing, vol. 17, nos. 4–5, pp. 443–455, 1991.

[14] R. K. Ahuja, K. C. Jha, J. B. Orlin, and D. Sharma, “Very large-scale neighborhood search for the quadratic assignment problem,” INFORMS Journal on Computing, 2007.

[15] P. M. Hahn, Y.-R. Zhu, M. Guignard, and J. M. Smith, “Exact solution of emerging quadratic assignment problems,” International Transactions in Operational Research, vol. 17, no. 5, pp. 525–552, 2010.

[16] R. Barták, M. A. Salido, and F. Rossi, “New trends in constraint satisfaction, planning, and scheduling: A survey,” The Knowledge Engineering Review, vol. 25, no. 3, pp. 249–279, 2010.

[17] S. Hartmann and D. Briskorn, “An updated survey of variants and extensions of the resource-constrained project scheduling problem,” European Journal of Operational Research, vol. 297, no. 1, pp. 1–14, 2022.

[18] V. Valls, A. Pérez, and S. Quintanilla, “A graph colouring model for assigning a heterogeneous workforce to a given schedule,” European Journal of Operational Research, vol. 90, no. 2, pp. 285–302, 1996.

[19] F. T. Leighton, “A graph coloring algorithm for large scheduling problems,” Journal of Research of the National Bureau of Standards, vol. 84, no. 6, pp. 489–506, 1979.

[20] “Constraint logic programming: A survey,” The Journal of Logic Programming, vols. 19–20, pp. 503–581, 1994.

[21] “A survey of constraint-based scheduling systems using an artificial intelligence approach,” Artificial Intelligence in Engineering, vol. 6, no. 2, pp. 58–73, 1991.

[22] R. Dechter, “Enhancement schemes for constraint processing: Backjumping, learning, and cutset decomposition,” Artificial Intelligence, vol. 41, no. 3, pp. 273–312, 1990.

[23] N. K. Mehta, “The application of a graph coloring method to an examination scheduling problem,” Interfaces, vol. 11, no. 5, pp. 57–65, 1981.

[24] S. K. Ogunkan, P. O. Idowu, and E. O. Oyeleye, “First Fit Algorithm: A graph coloring approach to conflict-free university course timetabling,” Asian Journal of Research in Computer Science, 2024.